

# Python 实现 SSA 智能麻雀搜索算法优化 XGBoost 回归模型(XGBRegressor 算法)项目实战

## 1. 项目背景

麻雀搜索算法(Sparrow Search Algorithm, SSA)是一种新型的群智能优化算法，在 2020 年提出，主要是受麻雀的觅食行为和反捕食行为的启发。

在麻雀觅食的过程中，分为发现者（探索者）和加入者（追随者），发现者在种群中负责寻找食物并为整个麻雀种群提供觅食区域和方向，而加入者则是利用发现者来获取食物。为了获得食物，麻雀通常可以采用发现者和加入者这两种行为策略进行觅食。种群中的个体会监视群体中其它个体的行为，并且该种群中的攻击者会与高摄取量的同伴争夺食物资源，以提高自己的捕食率。此外，当麻雀种群意识到危险时会做出反捕食行为。

本项目通过 SSA 麻雀搜索算法优化 XGBoost 回归模型。

## 2. 数据获取

本次建模数据来源于网络(本项目撰写人整理而成)，数据项统计如下：

| 编号 | 变量名称 | 描述  |
|----|------|-----|
| 1  | x1   |     |
| 2  | x2   |     |
| 3  | x3   |     |
| 4  | x4   |     |
| 5  | x5   |     |
| 6  | x6   |     |
| 7  | x7   |     |
| 8  | x8   |     |
| 9  | x9   |     |
| 10 | x10  |     |
| 11 | y    | 因变量 |

数据详情如下(部分展示)：

| x1    | x2    | x3    | x4    | x5    | x6    | x7    | x8    | x9    | x10   | y       |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|---------|
| -1.57 | -1.33 | -0.4  | 0.16  | 1.02  | -1.56 | -0.81 | 0.08  | -1.15 | 2.43  | -197.58 |
| -1.16 | -1.89 | -0.22 | 1.23  | -0.53 | 0.85  | -0.98 | 0.83  | -1.44 | 1.35  | -79.08  |
| -2.81 | 0.11  | -0.9  | -2.28 | -0.81 | -0.53 | 0.34  | -0.99 | 0.37  | 0.19  | -477.67 |
| 0.66  | -0.09 | -1.58 | -1.16 | 0.55  | 0.08  | 0.66  | -0.42 | 0.52  | 0.04  | -80.64  |
| 1.13  | -0.69 | -0.52 | -0.49 | 2.27  | 2.99  | 2.98  | -1.98 | -0.29 | 1.5   | 428.54  |
| 0.96  | 1.74  | -1.07 | -0.98 | -1.01 | -0.68 | -0.04 | 1.18  | -0.13 | 0.77  | 7.82    |
| 0.85  | -0.46 | -0.64 | 0.72  | -0.96 | 0.56  | 0.55  | 0.91  | 0.41  | -0.19 | 120.25  |
| 1.08  | 0.52  | -1.64 | -0.01 | -0.67 | -0.39 | -0.01 | -1.19 | -1.11 | -0.01 | -159.99 |
| -1.52 | 0.43  | 0.29  | -0.5  | 0.44  | -0.66 | -0.15 | 0.25  | 0.12  | 0.6   | -71.39  |
| -0.61 | 0.31  | 0.42  | -0.49 | 0.65  | 0.58  | 1.59  | 0.58  | -0.31 | -0.81 | 57.79   |

### 3. 数据预处理

#### 3.1 用 Pandas 工具查看数据

使用 Pandas 工具的 head() 方法查看前五数据：

```

      x1    x2    x3    x4    x5    x6    x7    x8    x9    x10      y
0 -1.57 -1.33 -0.40  0.16  1.02 -1.56 -0.81  0.08 -1.15  2.43 -197.58
1 -1.16 -1.89 -0.22  1.23 -0.53  0.85 -0.98  0.83 -1.44  1.35  -79.08
2 -2.81  0.11 -0.90 -2.28 -0.81 -0.53  0.34 -0.99  0.37  0.19 -477.67
3  0.66 -0.09 -1.58 -1.16  0.55  0.08  0.66 -0.42  0.52  0.04  -80.64
4  1.13 -0.69 -0.52 -0.49  2.27  2.99  2.98 -1.98 -0.29  1.50  428.54

```

关键代码：

```

# 用Pandas工具查看数据
print(data.head())

```

#### 3.2 数据缺失查看

使用 Pandas 工具的 info() 方法查看数据信息：

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0    x1      1000 non-null    float64
 1    x2      1000 non-null    float64
 2    x3      1000 non-null    float64
 3    x4      1000 non-null    float64
 4    x5      1000 non-null    float64
 5    x6      1000 non-null    float64
 6    x7      1000 non-null    float64
 7    x8      1000 non-null    float64
 8    x9      1000 non-null    float64
 9   x10     1000 non-null    float64
10    y       1000 non-null    float64
dtypes: float64(11)
memory usage: 86.1 KB
None

```

从上图可以看到，总共有 11 个变量，数据中无缺失值，共 1000 条数据。  
关键代码：

```

# 数据缺失值统计
print('*****')
print(data.info())

```

### 3.3 数据描述性统计

通过 Pandas 工具的 describe() 方法来查看数据的平均值、标准差、最小值、分位数、最大值。

|       | x1        | x2        | x3        | ... | x9        | x10       | y         |
|-------|-----------|-----------|-----------|-----|-----------|-----------|-----------|
| count | 1000.0000 | 1000.0000 | 1000.0000 | ... | 1000.0000 | 1000.0000 | 1000.0000 |
| mean  | 0.0131    | 0.0628    | 0.0600    | ... | -0.0390   | 0.0199    | 9.9503    |
| std   | 1.0134    | 0.9963    | 1.0603    | ... | 1.0192    | 1.0283    | 196.3264  |
| min   | -3.0700   | -2.9700   | -3.2800   | ... | -3.2900   | -2.9600   | -597.3200 |
| 25%   | -0.6825   | -0.6600   | -0.6200   | ... | -0.7300   | -0.7200   | -124.6425 |
| 50%   | 0.0200    | 0.1000    | 0.0800    | ... | -0.0300   | 0.0400    | 3.3050    |
| 75%   | 0.7100    | 0.7800    | 0.7825    | ... | 0.6300    | 0.7000    | 140.2725  |
| max   | 3.8700    | 2.7200    | 3.3000    | ... | 3.5500    | 3.6200    | 657.4100  |

[8 rows x 11 columns]

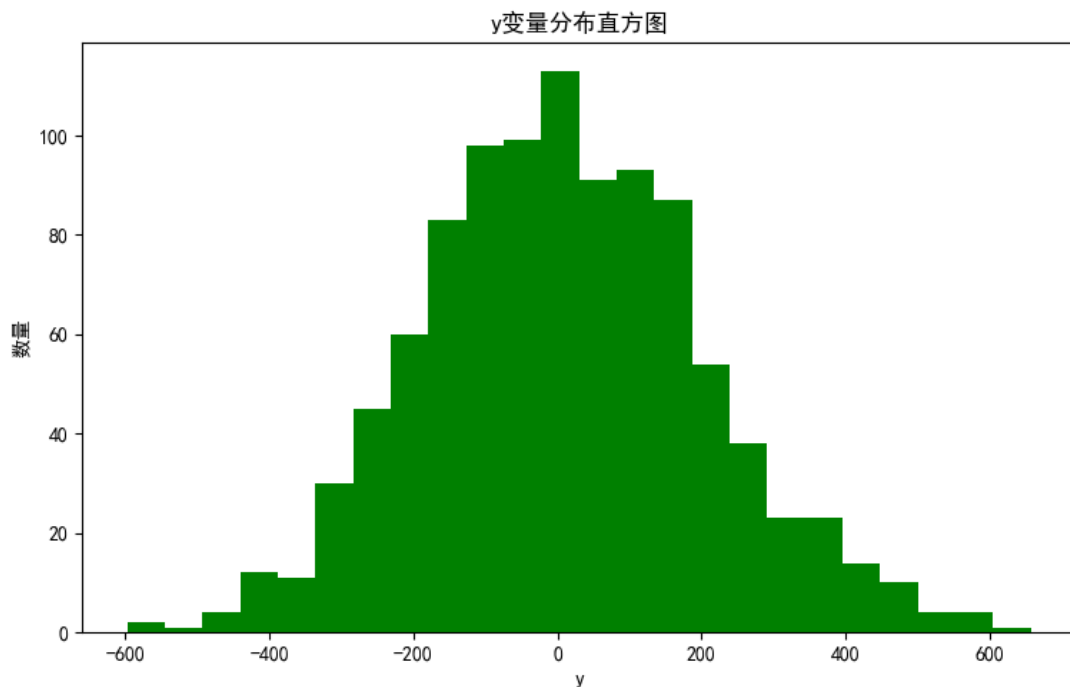
关键代码如下：

```
print('*****数据描述性统计分析*****')
print(data.describe().round(4)) # 保留4位小数点
```

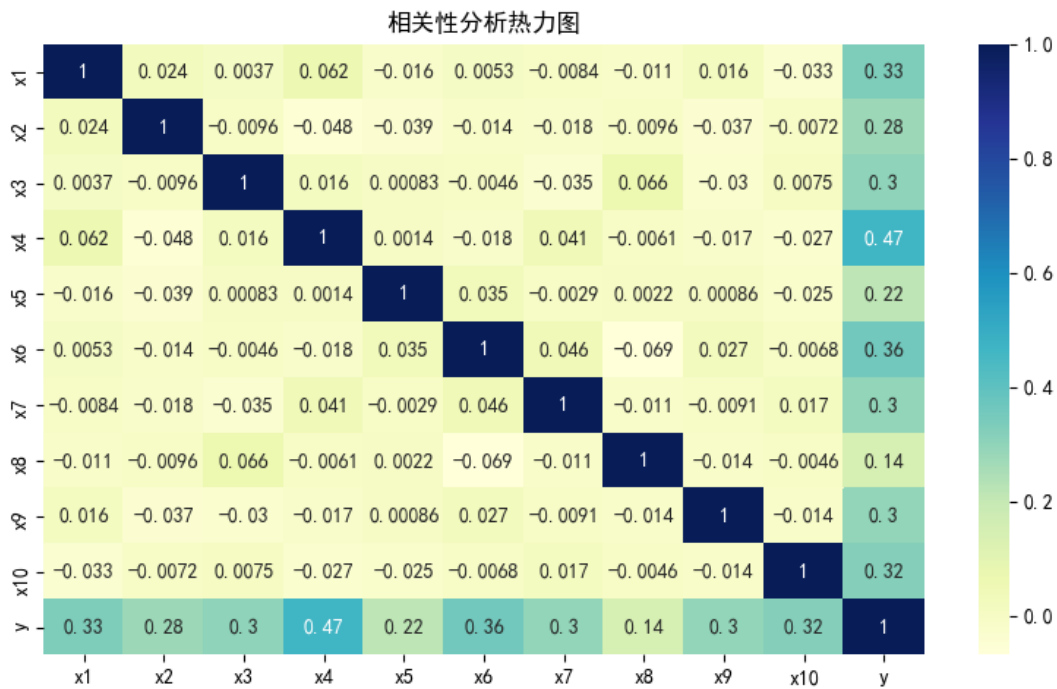
## 4. 探索性数据分析

### 4.1 y 变量分布直方图

用 Matplotlib 工具的 hist() 方法绘制直方图：



## 4.2 相关性分析



从上图中可以看到，数值越大相关性越强，正值是正相关、负值是负相关。

## 5. 特征工程

### 5.1 建立特征数据和标签数据

关键代码如下：

```
# 构建特征和标签
X = data.drop(columns=['y']) # 构建特征
y = data['y'] # 构建标签
```

### 5.2 数据集拆分

通过 `train_test_split()` 方法按照 80%训练集、20%验证集进行划分，关键代码如下：

```
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

6. 构建 SSA 麻雀搜索算法优化 XGBoost 回归模型

主要使用 SSA 麻雀搜索算法优化 XGBoost 算法，用于目标回归。

6.1 SSA 麻雀搜索算法寻找最优参数值

最优参数值：

```
-----SSA智能麻雀搜索算法优化XGBoost回归模型-最优结果展示-----
The best n_estimators is 1000
The best learning_rate is 0.1
```

6.2 最优参数构建模型

这里通过最优参数构建 XGBoost 回归模型：

| 模型名称         | 模型参数                                          |
|--------------|-----------------------------------------------|
| XGBoost 回归模型 | <code>n_estimators=best_n_estimators</code>   |
|              | <code>learning_rate=best_learning_rate</code> |

7. 模型评估

7.1 评估指标及结果

评估指标主要包括 R 方、均方误差、解释性方差、绝对误差等等。

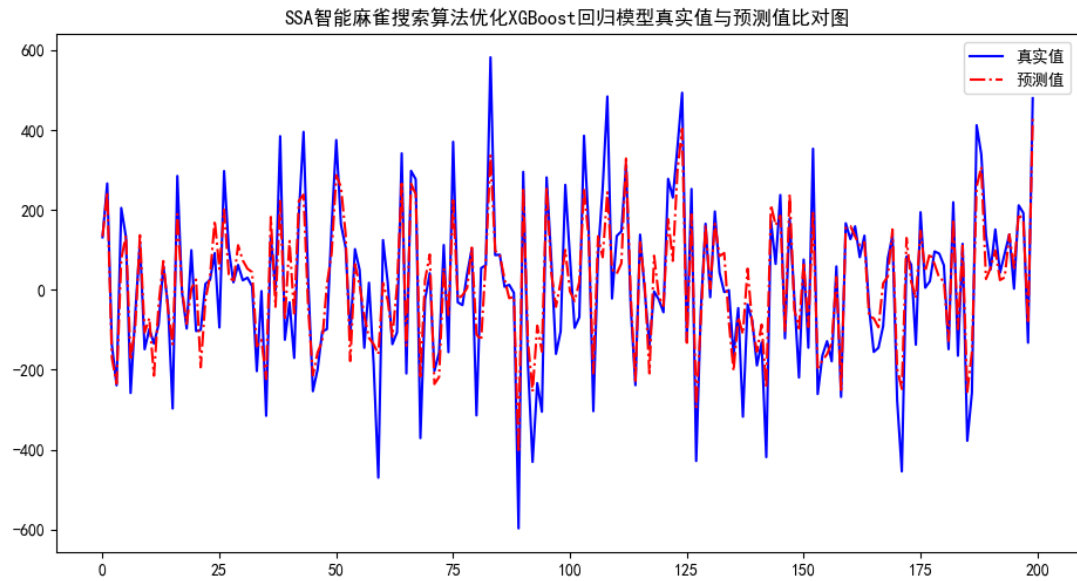
| 模型名称         | 指标名称  | 指标值       |
|--------------|-------|-----------|
| 测试集          |       |           |
| XGBoost 回归模型 | R 方   | 0.8201    |
|              | 均方误差  | 7399.8828 |
|              | 解释方差分 | 0.8211    |
|              | 绝对误差  | 66.2675   |

从上表可以看出，R 方分值为 0.8201，模型效果良好。

关键代码如下：

```
print('XGBoost回归模型-最优参数-R方值: {}'.format(round(r2_score(y_test, y_pred), 4)))
print('XGBoost回归模型-最优参数-均方误差: {}'.format(round(mean_squared_error(y_test, y_pred), 4)))
print('XGBoost回归模型-最优参数-可解释方差值: {}'.format(round(explained_variance_score(y_test, y_pred), 4)))
print('XGBoost回归模型-最优参数-平均绝对误差: {}'.format(round(mean_absolute_error(y_test, y_pred), 4)))
```

## 7.2 真实值与预测值对比图



从上图可以看出真实值和预测值波动基本一致，模型效果良好。

## 8. 结论与展望

综上所述，本文采用了 SSA 麻雀搜索算法寻找 XGBoost 算法的最优参数值来构建回归模型，最终证明了我们提出的模型效果良好。此模型可用于日常产品的建模工作。